Miniprojekt 1

Numerical Scientific Computing

Sebastian Pabisz Frølund

2. marts 2026

STUDENTER RAPPORT

AALBORG UNIVERSITY

Copyright © Aalborg Universitet 2026

1 Specs

	Specs
CPU	i7-1360P
GPU	Intel(R) Iris(R) Xe Graphics
Ram	16 GB

Tabel 1.1: Specs of the computer used for this miniproject

2 Profiling

To understand where the code spends most of its time a profile was constructed. Firstly a function-level profile was constructed for the naive version of the Mandelbrot set generator, which yielded the results below.

ncall	tottime	cumtime	function
1	0.226	2.411	mandelbrot_set_naive
262144	1.718	2.185	mandelbrot_point
5698796	0.467	0.467	{built-in method builtins.abs}

Tabel 2.1: Naive Mandelbrot

From the function-level profile of the naive implementation given in Tabel 2.1 it is seen that the function `mandelbrot_point` is called a lot of times and the abs function is called even more. This results in the code spending the majority of its time in these functions, which was expected as the are a part of the inner most loop and thus needs to be called a lot of times. To further inspect where the naive implementation spends its time a line-level profile was also constructed which is visualized in Tabel 2.2 below.

Line #	Hits	Time	Per Hit	%Time	Line Contents
3	–	–	–	0.0	@profile
4	–	–	–	0.0	def mandelbrot_set_naive_lineprofile(*args):
5	1	97.8	97.8	0.0	x = np.linspace(xmin, xmax, width)
6	1	25.5	25.5	0.0	y = np.linspace(ymin, ymax, height)
7	1	18.3	18.3	0.0	`result = np.zeros(args*,kwargs**)`
8	513	55.3	0.1	0.0	for i in range(height):
9	262656	23198.6	0.1	0.7	for j in range(width):
10	262144	32288.0	0.1	10.1	c = x[j] + 1j * y[i]
11	262144	23580.8	0.1	0.7	z = 0
12	5748674	52037.1	0.0	16.3	for n in range(max_iter):
13	5689766	111291.2	0.0	35.2	if abs(z) > 2:
14	217266	44478.8	0.2	1.4	result[i, j] = n
15	217266	20540.3	0.1	0.6	break
16	550481	110026.4	0.2	34.5	z = z*z + c
17	–	–	–	0.0	else:
18	44878	12485.7	0.3	0.4	result[i, j] = max_iter
19	1	2.0	2.0	0.0	return result

Tabel 2.2: Line-by-line profiling of `mandelbrot_set_naive_lineprofile`

From the line-level profile the problem is further confirmed that the inner loop is the problem. In

Tabel 2.2 it is once again seen that the abs function takes a lot of time and more precisely 35.2% of the total run time. Furthermore line which updates the complex variable z takes 34.5%. Both of these lines are placed in the inner most loop confirming the initial function-level profile which located the problem in the inner loop. The initial function-level profile only indicated the problem by the abs function but with the line-level profile it is evident that the inner most loop is the problem as it is here where the code spends $\sim 70\%$ of its time. These results will now be compared to the vectorized version which also was profiled on a function level. The results of the function-level profile is given below.

ncalls	tottime	cumtime	function
1	0.296	0.298	mandelbrot_set_vectorized
1	0.000	0.001	meshgrid

Tabel 2.3: Vectorized Mandelbrot

From Tabel 2.3 it is evident that the vectorized has much fewer calls where nothing is really called multiple times. Thus the vectorized version solves the major problem with the naive implementation as it omits the abusive use of nested loops and is able to perform calculations on everything all at once instead of iterating over every pixel one at a time. Thus the vectorized implementation results in greatly improved performance compared to the naive implementation.

3 Optimization Overview

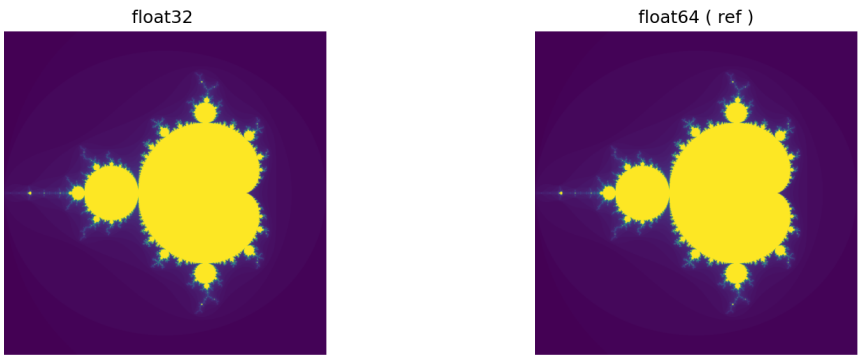
This chapter gives an overview of the timings of the different implementations developed so far in the course. The results are given in Tabel 3.1 below.

Implementation Type	Time (seconds)
Naive implementation	2.9462
Vectorized implementation	1.1102
Hybrid Numba implementation	1.5372
Numba64 implementation	0.0529
Numba32 implemetation	0.0481

Tabel 3.1: Timing table of the four different implementations.

In Tabel 3.1 the expected results are seen, which are that the vectorized version is better than the naive because what was discussed earlier. Furthermore Numba improves on the vectorized version due to compilation and structural improvements. It is though worth noticing that the hybrid version does not outperform the vectorized one since nested loops are still present in ordinary python and thus elongates the runtime.

Finally there is the discussion of data type optimization. On my computer the difference between single and double precision for the Numba implementation is negligible. It is literally 50-50 which one executes the code faster, in the table only one realization of a benchmark with warmup run and then the median of 5 runs is shown. In reality they just always execute in ~ 0.05s and which one is faster is unpredictable. I expect this is a hardware thing? I have given up on trying to edit my code because nothing i have tried gave any different output and i ended up splitting the function into to and directly set one to `np.float64` and `np.complex128` and the other to `np.float32` and `np.complex128`, but this still did not result in different runtimes. They did however give different outputs which are shown in Figur 3.1 below.



Figur 3.1: Comparison picture

From Figur 3.1 there is no immediate difference to spot, and thus the total amount of different pixels and the maximum difference were found and listed in Tabel 3.2 below.

	Difference
Max	32
Number of pixels	442

Tabel 3.2: Data type comparison

From these numbers it is seen that the difference in pixels can be quite substantial as a difference of 32 is huge, when only values between 0 and 100 can be taken. However, only a small fraction of the pixels in the 1024×1024 image differ which suggests that `np.float32` provides sufficient precision for these calculations. This is further supported by the minimal visual differences between the images. Thus a general recommendation for these calculations would be to use `np.float32`, but the decision is probably hardware dependent as my machine does not seem to experience any runtime improvements by using `np.float32` and therefor i might as well use `np.float64` to get the extra precision at zero runtime cost. (Unless i have royally screwed something in my code but i really can not find any problems with it)